

Topological Deep Learning

Elvin Isufi

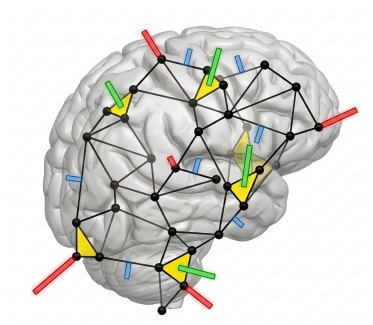
Intelligent Systems Department

May 15, 2026

Motivation

Brain data:

- ▶ **Topological signal:** activation levels for
 - ⇒ single neurons (nodes)
 - ⇒ groups of nodes (edges, triangles)
 - ⇒ hierarchical relationships
- ▶ **Complex tasks:**
 - ⇒ noisy measurements
 - ⇒ non-linear relationships
 - ⇒ e.g., predict the activation of a region, identify new groups of neurons that co-activate, etc.



Motivation

Option 1: Graph Deep Learning

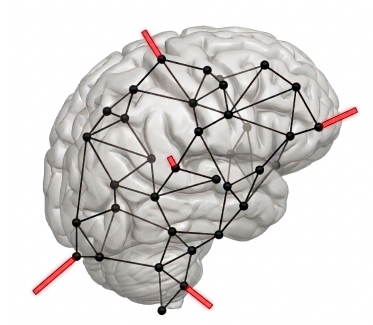
► Pros:

⇒ can learn complex, non-linear functions

► Cons:

⇒ can only process graph information

⇒ ignores topological inductive biases



Motivation

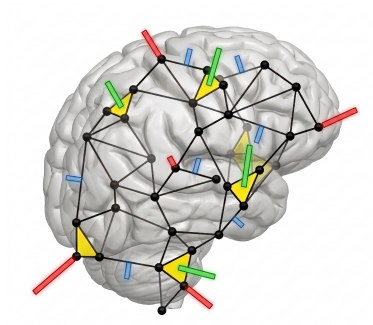
Option 2: Topological Convolutional Filtering

► Pros:

- ⇒ can leverage the simplicial complex topology and signals
- ⇒ can solve topological tasks

► Cons:

- ⇒ can only learn simple, linear functions



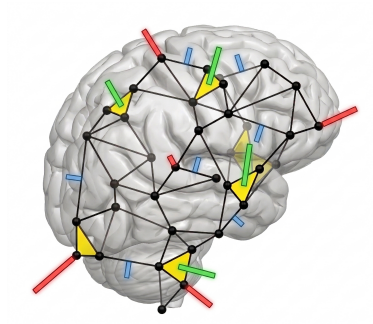
Motivation

Option 3: Topological Deep Learning

► Pros:

- ⇒ can learn complex, non-linear functions
- ⇒ can leverage the simplicial complex topology and signals
- ⇒ can solve topological tasks

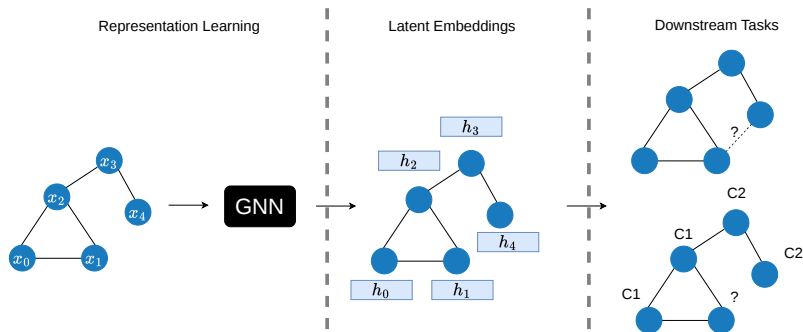
Best of both worlds!



Graph Representation Learning

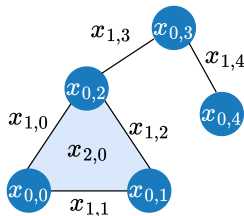
► Objectives

- ⇒ Leverage invariances, equivariances and inductive biases of the **graph structure**
- ⇒ Learn **complex, non-linear functions**
- ⇒ Produce **informative representations** in a latent space
- ⇒ Solve **downstream tasks**



Graph Representation Learning

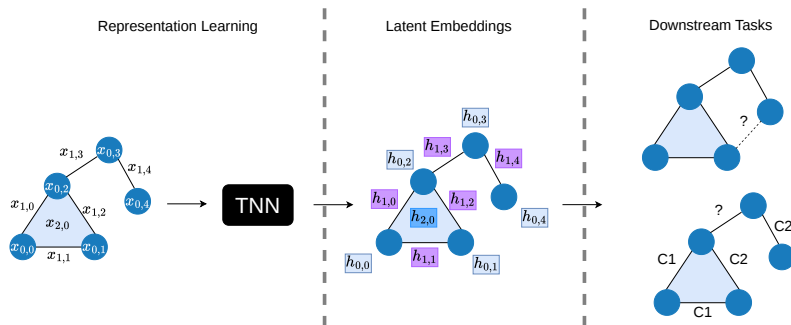
- **Limitations:** when the signal is topological
 - ⇒ Signals are not only on nodes but on **all simplices**
 - ⇒ Interactions occur among **more than two nodes**
 - ⇒ There are **hierarchies** among simplices
 - ⇒ There are **new invariances and equivariances**: e.g., orientation



Topological Representation Learning

Core objective: learn representations from topological signals

- ▶ Leverage structure **invariants**
- ▶ Learn **expressive** mapping from data



Outline: What is this Lecture About?

► How to perform deep learning with topological structures and signals?

⇒ Topological convolutional networks

⇒ Topological attention networks

⇒ Learning from joint signals

⇒ Self-supervision with topologies

Topological Convolutional Networks

Simplicial Convolutional Filters

- The simplicial convolutional filter acts as my mapping: $\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) \rightarrow \mathbf{f}_o$

$$\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) = \left(h\mathbf{I} + \sum_{l_1=1}^{L_1} h_{l,l_1} (\mathbf{L}_{1,\ell})^{l_1} + \sum_{l_2=1}^{L_2} h_{u,l_2} (\mathbf{L}_{1,u})^{l_2} \right) \mathbf{f}$$

- Optimal regressor restricted to filter class

$$\mathcal{H}^* = \arg \min_{\mathcal{H}} \sum_{\mathbf{f} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}))$$

Simplicial Convolutional Filters

- The simplicial convolutional filter acts as my mapping: $\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) \rightarrow \mathbf{f}_o$

$$\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) = \left(h\mathbf{I} + \sum_{l_1=1}^{L_1} h_{l,l_1} (\mathbf{L}_{1,\ell})^{l_1} + \sum_{l_2=1}^{L_2} h_{u,l_2} (\mathbf{L}_{1,u})^{l_2} \right) \mathbf{f}$$

- Optimal regressor restricted to filter class

$$\mathcal{H}^\star = \arg \min_{\mathcal{H}} \sum_{\mathbf{f} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}))$$

- **Advantages**
 - ⇒ Useful for linear, principled and tractable processing
 - ⇒ Mostly used for signal-based regression tasks

Simplicial Convolutional Filters

- ▶ The simplicial convolutional filter acts as my mapping: $\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) \rightarrow \mathbf{f}_o$

$$\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) = \left(h\mathbf{I} + \sum_{l_1=1}^{L_1} h_{l,l_1} (\mathbf{L}_{1,\ell})^{l_1} + \sum_{l_2=1}^{L_2} h_{u,l_2} (\mathbf{L}_{1,u})^{l_2} \right) \mathbf{f}$$

- ▶ Optimal regressor restricted to filter class

$$\mathcal{H}^* = \arg \min_{\mathcal{H}} \sum_{\mathbf{f} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}))$$

- ▶ **Advantages**

- ⇒ Useful for linear, principled and tractable processing
- ⇒ Mostly used for signal-based regression tasks

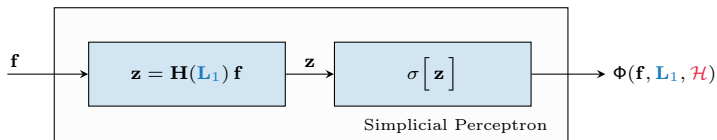
- ▶ **Limitations**

- ⇒ **Linear** expressive power
- ⇒ Suffers in **classification** and topology-related tasks: closure prediction, simplicial classification

Simplicial Perceptron

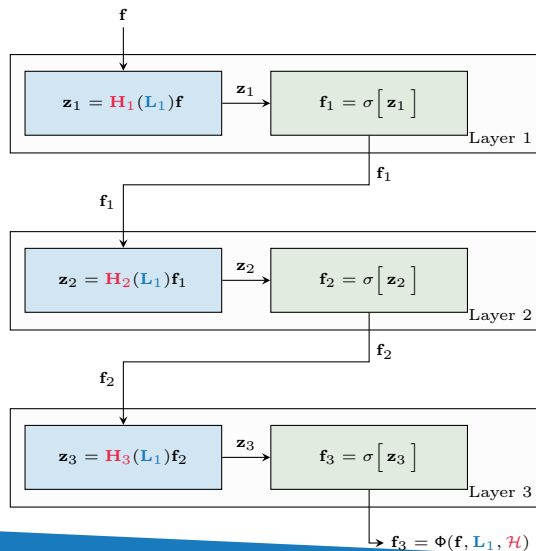
- Increase the **expressivity** of filters via simplicial **perceptron**

$$\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H}) = \sigma \left[\mathbf{H}(\mathbf{L}_1) \mathbf{f} \right]$$



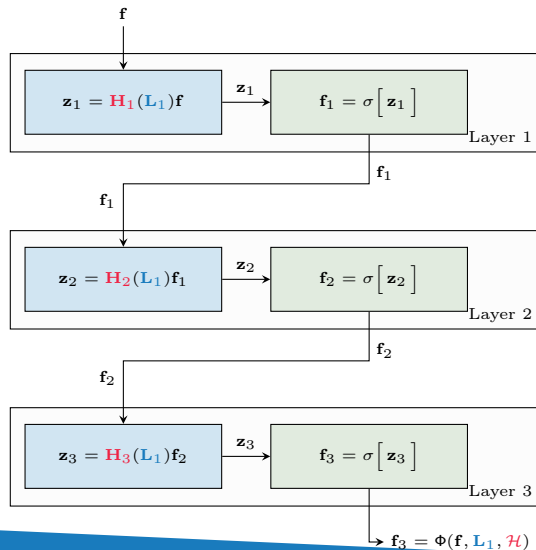
- **simplicial**-inducted learning – $\mathbf{L}_1$
 - $\Rightarrow$ parameters $\mathcal{H}$ independent on the problem dimension
 - $\Rightarrow$ transferable – inductive learning
 - $\Rightarrow \Phi(\cdot)$ is local
 - $\Rightarrow$ learn **non-linear** mapping

Simplicial Convolutional Neural Networks



- Hierarchical representation by cascading simplicial perceptrons

Simplicial Convolutional Neural Networks



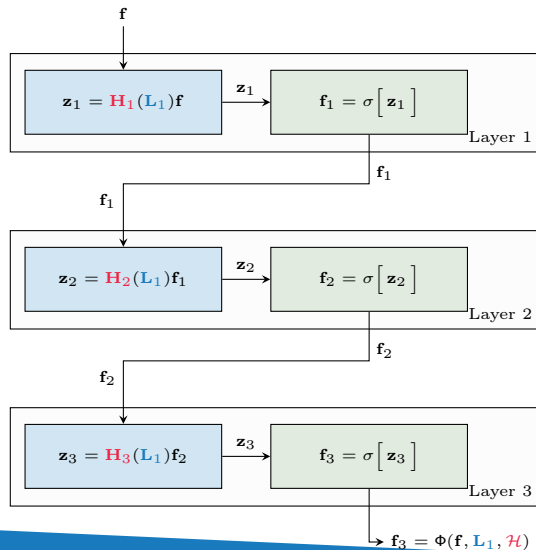
► **Hierarchical** representation by cascading simplicial perceptrons

► Layer ℓ propagation rule

$$\mathbf{f}_\ell = \sigma[\mathbf{z}_\ell] = \sigma\left[\mathbf{H}_\ell(\mathbf{L}_1)\mathbf{f}_{\ell-1}\right]$$

$\Rightarrow$ Trainable parameters: $\mathcal{H} = [\mathcal{H}_1, \dots, \mathcal{H}_L]$

Simplicial Convolutional Neural Networks



- **Hierarchical** representation by cascading simplicial perceptrons

- Layer ℓ propagation rule

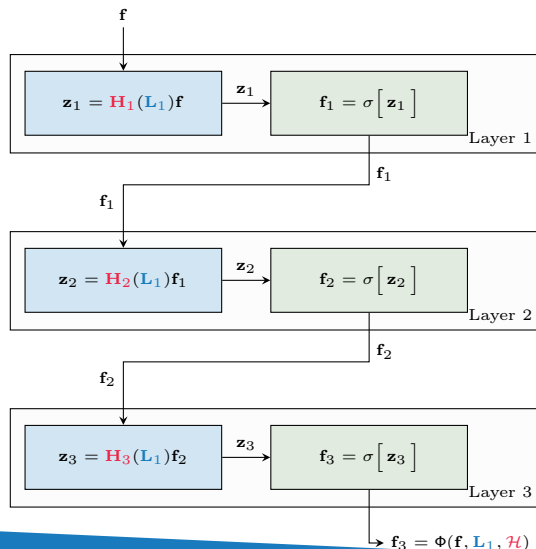
$$\mathbf{f}_\ell = \sigma[\mathbf{z}_\ell] = \sigma\left[\mathbf{H}_\ell(\mathbf{L}_1)\mathbf{f}_{\ell-1}\right]$$

$\Rightarrow$ Trainable parameters: $\mathcal{H} = [\mathcal{H}_1, \dots, \mathcal{H}_L]$

- **Learn** Optimal **tensor** $\mathcal{H}^* = (\mathcal{H}_1^*, \mathcal{H}_2^*, \mathcal{H}_3^*)$ as

$$\mathcal{H}^* = \arg \min_{\mathcal{H}} \sum_{f \in \mathcal{T}} \mathcal{L}\left(\Phi(f, \mathbf{L}_1, \mathcal{H})\right)$$

Simplicial Convolutional Neural Networks



- **Hierarchical** representation by cascading simplicial perceptrons

- Layer ℓ propagation rule

$$\mathbf{f}_\ell = \sigma[\mathbf{z}_\ell] = \sigma\left[\mathbf{H}_\ell(\mathbf{L}_1)\mathbf{f}_{\ell-1}\right]$$

$\Rightarrow$ Trainable parameters: $\mathcal{H} = [\mathcal{H}_1, \dots, \mathcal{H}_L]$

- Learn Optimal tensor $\mathcal{H}^* = (\mathcal{H}_1^*, \mathcal{H}_2^*, \mathcal{H}_3^*)$ as

$$\mathcal{H}^* = \arg \min_{\mathcal{H}} \sum_{\mathbf{f} \in \mathcal{T}} \mathcal{L}\left(\Phi(\mathbf{f}, \mathbf{L}_1, \mathcal{H})\right)$$

- Optimization is over parameter tensor only. **Topology** $\mathbf{L}_1$ is given

$\Rightarrow$ Relational inductive bias

MIMO Simplicial Convolutional Neural Networks

- The SCNN we saw so far processes a **single input** to a **single output**

$$\mathbf{f}_\ell = \sigma \left[\left(\mathbf{h} \mathbf{I} + \sum_{l_1=1}^{L_1} \mathbf{h}_{\ell, l_1} (\mathbf{L}_{1, \ell})^{l_1} + \sum_{l_2=1}^{L_2} \mathbf{h}_{u, l_2} (\mathbf{L}_{1, u})^{l_2} \right) \mathbf{f}_{\ell-1} \right]$$

MIMO Simplicial Convolutional Neural Networks

- The SCNN we saw so far processes a **single input** to a **single output**

$$\mathbf{f}_\ell = \sigma \left[\left(h\mathbf{I} + \sum_{l_1=1}^{L_1} h_{\ell,l_1} (\mathbf{L}_{1,\ell})^{l_1} + \sum_{l_2=1}^{L_2} h_{u,l_2} (\mathbf{L}_{1,u})^{l_2} \right) \mathbf{f}_{\ell-1} \right]$$

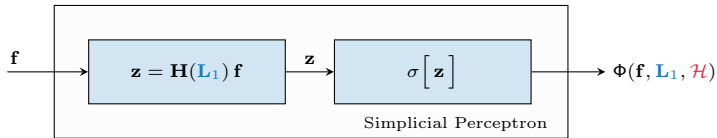
- We can further increase the expressive power by using a **filter bank** of $F \times G$ filters

$$\mathbf{F}_\ell = \sigma \left[\mathbf{F}_{\ell-1} \mathbf{W} + \sum_{l_1=1}^{L_1} (\mathbf{L}_{1,\ell})^{l_1} \mathbf{F}_{\ell-1} \mathbf{W}_{\ell,l_1} + \sum_{l_2=1}^{L_2} (\mathbf{L}_{1,u})^{l_2} \mathbf{F}_{\ell-1} \mathbf{W}_{\ell,l_2} \right]$$

$\Rightarrow \mathbf{W}, \mathbf{W}_{\ell,l_1}, \mathbf{W}_{u,l_2}$ collect the multiple-input multiple-output **filter bank coefficients**

Equivariances

Informal

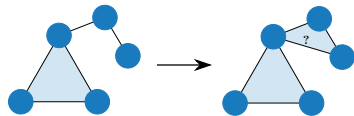


Permutation equivariance. The SCNN is equivariant to permutations.

Orientation equivariance. The SCNN is equivariant to orientations **only** if nonlinearity $\sigma(\cdot)$ is **odd**.

Numerical Comparison

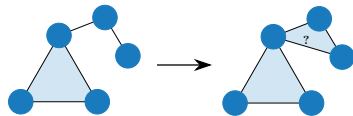
- Closure prediction
⇒ Predict the creation of social groups



Numerical Comparison

► Closure prediction

⇒ Predict the creation of social groups



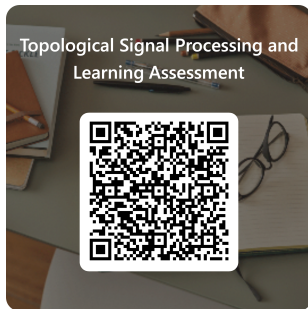
Methods	2-Simplex	3-Simplex
Harm. Mean (Benson et al., 2018)	62.8 $\pm$ 2.7	63.6 $\pm$ 1.6
MLP	68.5 $\pm$ 1.6	69.0 $\pm$ 2.2
GF (Sandryhaila & Moura, 2013)	78.7 $\pm$ 1.2	83.9 $\pm$ 2.3
SCF (Yang et al., 2022b)	92.6 $\pm$ 1.8	94.9 $\pm$ 1.0
CF-SC (Isufi & Yang, 2022)	96.9 $\pm$ 0.8	97.9 $\pm$ 0.7
GNN (Gama et al., 2020a)	93.9 $\pm$ 1.0	96.6 $\pm$ 0.5
SNN (Ebli et al., 2020)	92.0 $\pm$ 1.8	95.1 $\pm$ 1.2
PSNN (Roddenberry et al., 2021)	95.6 $\pm$ 1.3	98.1 $\pm$ 0.5
SCNN (Yang et al., 2022a)	96.5 $\pm$ 1.5	98.3 $\pm$ 0.4
Bunch (Bunch et al., 2020)	98.0 $\pm$ 0.5	98.5 $\pm$ 0.5
SCCNN	98.4$\pm$0.5	99.4$\pm$0.3

- Simplicial convolutions **compare** favorably with message passing solutions while providing a principled tractability

Learning Activity: What Makes SCNNs “Deep”?

Simplicial filters are linear operators constrained by the Hodge Laplacian. SCNNs increase expressivity by stacking nonlinear simplicial perceptrons, while still preserving the topological inductive bias.

- ▶ **Q6.1:** Which statement best explains why SCNNs can outperform simplicial convolutional filters on tasks like classification or closure prediction?
- ▶ **Instructions**
 - ⇒ Scan the QR code to open the activity
 - ⇒ Work individually first
 - ⇒ Then discuss your answer in pairs or groups of 3
 - ⇒ Submit your final answer in Microsoft Forms
- ▶ **Logistics**
 - ⇒ **Time:** 8 min
 - ⇒ **Goal:** Understand how nonlinearity and depth change the expressive power of topological filters
 - ⇒ **Product:** One submitted response per student + justification to your peers

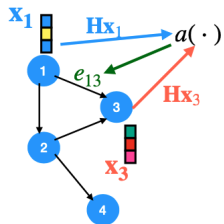


<https://forms.office.com/>

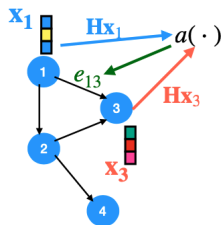
Topological Attention Networks

Graph Attention

- Idea: Learn **parametric** edge weights from the node features



Graph Attention

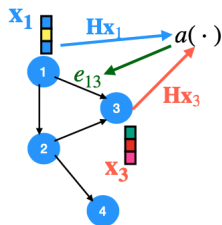


- **Idea:** Learn **parametric** edge weights from the node features
- A function $a : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ computes **attention** coefficients

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j)$$

- e_{ij} : **importance** of node j 's feature to node i
 - $\Rightarrow \mathbf{x}_i \in \mathbb{R}^F$ are the features of node i
 - $\Rightarrow \mathbf{H} \in \mathbb{R}^{F \times F}$ is the linear transformation matrix

Graph Attention

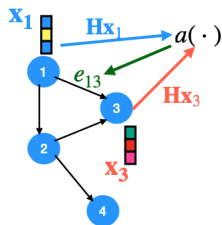


- **Idea:** Learn **parametric** edge weights from the node features
- A function $a : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ computes **attention** coefficients

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j)$$

- e_{ij} : **importance** of node j 's feature to node i
 - $\Rightarrow \mathbf{x}_i \in \mathbb{R}^F$ are the features of node i
 - $\Rightarrow \mathbf{H} \in \mathbb{R}^{F \times F}$ is the linear transformation matrix
- Only compute e_{ij} for neighbors $j \in \mathcal{N}_i$
 - $\Rightarrow$ Preserves the node **locality**
 - $\Rightarrow$ edge existence is the inductive bias not weight

Graph Attention



- **Idea:** Learn **parametric** edge weights from the node features
- A function $a : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ computes **attention** coefficients

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j)$$

- e_{ij} : **importance** of node j 's feature to node i
 - $\Rightarrow \mathbf{x}_i \in \mathbb{R}^F$ are the features of node i
 - $\Rightarrow \mathbf{H} \in \mathbb{R}^{F \times F}$ is the linear transformation matrix
- Only compute e_{ij} for neighbors $j \in \mathcal{N}_i$
 - $\Rightarrow$ Preserves the node **locality**
 - $\Rightarrow$ edge existence is the inductive bias not weight
- Perform **local normalization**

$$\alpha_{ij} = \text{softmax}(e_{ij})$$

$\Rightarrow$ Induces sparsity in the parameters learned

Math Behind Attention Score

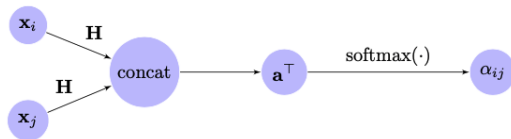
- General function $a(\cdot)$

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j) \xrightarrow{\text{normalization}} \alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i \cup i} \exp(e_{ik})}$$

Math Behind Attention Score

- General function $a(\cdot)$

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j) \xrightarrow{\text{normalization}} \alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i \cup i} \exp(e_{ik})}$$



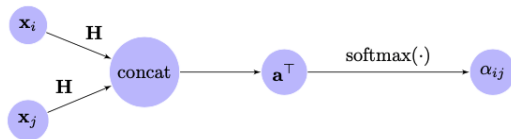
- Linear transform $\mathbf{a}^\top$

$$\alpha_{ij} = \mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \parallel \mathbf{H}\mathbf{x}_j]$$

Math Behind Attention Score

► General function $a(\cdot)$

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j) \xrightarrow{\text{normalization}} \alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i \cup i} \exp(e_{ik})}$$



► Linear transform $\mathbf{a}^\top$

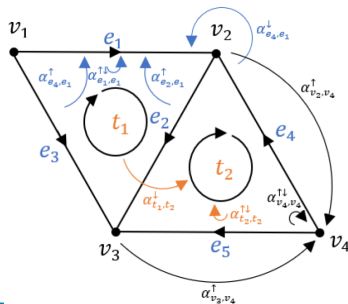
$$\alpha_{ij} = \mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \| \mathbf{H}\mathbf{x}_j] \xrightarrow{\text{normalization}} \alpha_{ij} = \frac{\exp(\mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \| \mathbf{H}\mathbf{x}_j])}{\sum_{k \in \mathcal{N}_i \cup i} \exp(\mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \| \mathbf{H}\mathbf{x}_k])}$$

$\Rightarrow$ $\|$ - concatenation operator

$\Rightarrow$ Performed for all edges $(i, j) \in \mathcal{E}$

Simplicial Attention

- ▶ Simplicial convolutional networks exploit **prior** knowledge of the Hodge Laplacian **structure** $\mathbf{L}_1$
- ▶ Whenever prior knowledge is **inaccurate** for the given task, better performance can be obtained by **learning** the relative **importance** of data **relations** over different neighborhoods of the simplicial complex
- ▶ **Simplicial Attention**: Learn the coefficients of upper and lower Laplacians $\mathbf{L}_{1,l}$ and $\mathbf{L}_{1,u}$ in a task-oriented fashion, for a given topological structure



Simplicial Attention Networks

- **Step 1:** Applies learnable linear transforms to input $\mathbf{F}_\ell$:

$$\mathbf{h}_{\ell,l_2,i,u} = [\mathbf{F}_\ell]_i \mathbf{H}_{\ell,l_2,u}, \quad \mathbf{h}_{\ell,l_1,i,l} = [\mathbf{F}_\ell]_i \mathbf{H}_{\ell,l_1,l}$$

Simplicial Attention Networks

- **Step 1:** Applies learnable linear transforms to input $\mathbf{F}_\ell$:

$$\mathbf{h}_{\ell,l_2,i,u} = [\mathbf{F}_\ell]_i \mathbf{H}_{\ell,l_2,u}, \quad \mathbf{h}_{\ell,l_1,i,l} = [\mathbf{F}_\ell]_i \mathbf{H}_{\ell,l_1,l}$$

- **Step 2:** Concatenate transformed features:

$$\mathbf{h}_{\ell,i,u} = \{\mathbf{h}_{\ell,l_2,i,u}\}, \quad \mathbf{h}_{\ell,i,l} = \{\mathbf{h}_{\ell,l_1,i,l}\}$$

Simplicial Attention Networks

- **Step 1:** Applies learnable linear transforms to input $\mathbf{F}_\ell$:

$$\mathbf{h}_{\ell,l_2,i,u} = [\mathbf{F}_\ell]_i \mathbf{H}_{\ell,l_2,u}, \quad \mathbf{h}_{\ell,l_1,i,l} = [\mathbf{F}_\ell]_i \mathbf{H}_{\ell,l_1,l}$$

- **Step 2:** Concatenate transformed features:

$$\mathbf{h}_{\ell,i,u} = \{\mathbf{h}_{\ell,l_2,i,u}\}, \quad \mathbf{h}_{\ell,i,l} = \{\mathbf{h}_{\ell,l_1,i,l}\}$$

- **Step 3:** Attention scores at every layer ℓ are computed for each pair of edges i, j belonging to the upper and lower connectivity as:

$$\begin{aligned} e_{\ell,i,j,u} &= a_{\ell,u}(\mathbf{h}_{\ell,i,u}, \mathbf{h}_{\ell,j,u}) && \text{for } j \in \mathcal{N}_{i,u} \\ e_{\ell,i,j,l} &= a_{\ell,l}(\mathbf{h}_{\ell,i,l}, \mathbf{h}_{\ell,j,l}) && \text{for } j \in \mathcal{N}_{i,l} \end{aligned}$$

where $a_{\ell,u}$ and $a_{\ell,l}$ are upper and lower attentional mechanisms

Simplicial Attention Mechanisms

- The upper and lower attentional coefficients are computed as one-layer feedforward neural network and then normalized via softmax:

$$\alpha_{\ell,i,j,u} = \text{softmax}\left([\mathbf{h}_{\ell,i,u} \parallel \mathbf{h}_{\ell,j,u}]^\top \mathbf{a}_{\ell,u}\right), \quad \text{for } j \in \mathcal{N}_{i,u}$$
$$\alpha_{\ell,i,j,l} = \text{softmax}\left([\mathbf{h}_{\ell,i,l} \parallel \mathbf{h}_{\ell,j,l}]^\top \mathbf{a}_{\ell,l}\right) \quad \text{for } j \in \mathcal{N}_{i,l}$$

$\alpha_{\ell,i,j,u}$ and $\alpha_{\ell,i,j,l}$ are the coefficients of the **upper** and **lower** attentional Laplacians $\mathbf{L}_{\ell,u}$ and $\mathbf{L}_{\ell,l}$

Simplicial Attention Mechanisms

- The upper and lower attentional coefficients are computed as one-layer feedforward neural network and then normalized via softmax:

$$\alpha_{\ell,i,j,u} = \text{softmax}\left([\mathbf{h}_{\ell,i,u} \parallel \mathbf{h}_{\ell,j,u}]^\top \mathbf{a}_{\ell,u}\right), \quad \text{for } j \in \mathcal{N}_{i,u}$$
$$\alpha_{\ell,i,j,l} = \text{softmax}\left([\mathbf{h}_{\ell,i,l} \parallel \mathbf{h}_{\ell,j,l}]^\top \mathbf{a}_{\ell,l}\right) \quad \text{for } j \in \mathcal{N}_{i,l}$$

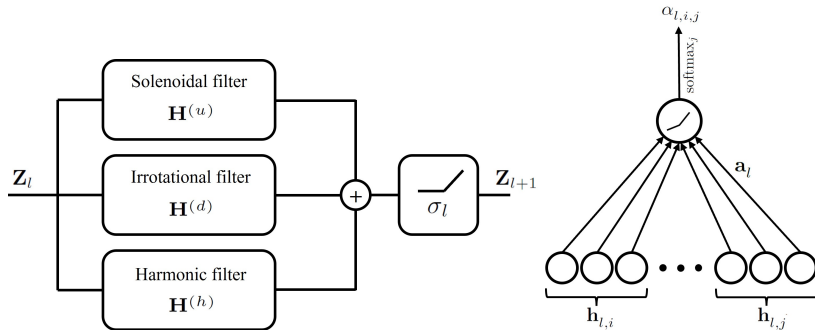
$\alpha_{\ell,i,j,u}$ and $\alpha_{\ell,i,j,l}$ are the coefficients of the **upper** and **lower** attentional Laplacians $\mathbf{L}_{\ell,u}$ and $\mathbf{L}_{\ell,l}$

- Output of a Simplicial Attention Network (SAN) layer:

$$\mathbf{z}_{\ell+1} = \sigma_\ell \left(\sum_p \mathbf{L}_{\ell,l}^p \mathbf{F}_\ell \mathbf{W}_{\ell,p,l} + \sum_p \mathbf{L}_{\ell,u}^p \mathbf{F}_\ell \mathbf{W}_{\ell,p,u} + \mathbf{K}_\ell \mathbf{F}_\ell \mathbf{W}_{\ell,h} \right)$$

where the layer parameters are: $\mathbf{W}_{\ell,p,l}$, $\mathbf{W}_{\ell,p,u}$, $\mathbf{W}_{\ell,h}$, $\mathbf{a}_{\ell,l}$, $\mathbf{a}_{\ell,u}$

Simplicial Attention Networks

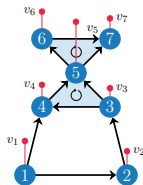


- ▶ SANs are made of the composition of three filtering operations plus a non linearity
- ▶ The attention mechanism is designed as a simple feedforward neural network with LeakyReLU activation function

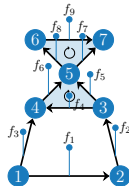
Learning from Joint Signals

Learn from Joint Signals

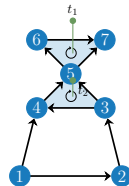
- For **didactical** purposes we focused at a **single** level signal $\Rightarrow$ e.g., **edge** flows
- It is common however to encounter joint signals and leverage also **level-to-level** structure
 $\Rightarrow$ Hodge decomposition hints towards this



Node signals



Edge signals



Triangle signals

- Simplicial complex signals

$$\mathbf{x} = \begin{bmatrix} \mathbf{v} \\ \mathbf{f} \\ \mathbf{t} \end{bmatrix}$$

$\Rightarrow$ We want to learn from $\mathbf{x}$ and the **whole** simplicial complex

Processing Joint Signals

- Dirac operator and the simplicial complexes signals

$$\mathbf{D} = \mathbf{D}_l + \mathbf{D}_u = \begin{bmatrix} \mathbf{0} & \mathbf{B}_1 & \mathbf{0} \\ \mathbf{B}_1^\top & \mathbf{0} & \mathbf{B}_2 \\ \mathbf{0} & \mathbf{B}_2^\top & \mathbf{0} \end{bmatrix}$$

$\Rightarrow$ where

$$\mathbf{D}_l = \begin{bmatrix} \mathbf{0} & \mathbf{B}_1 & \mathbf{0} \\ \mathbf{B}_1^\top & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad \mathbf{D}_u = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{B}_2 \\ \mathbf{0} & \mathbf{B}_2^\top & \mathbf{0} \end{bmatrix}$$

$\Rightarrow$ The Dirac operator satisfying

$$\mathbf{D}^2 = \mathcal{L} = \begin{bmatrix} \mathbf{L}_0 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{L}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{L}_2 \end{bmatrix}$$

Topological Filters with Dirac

- Simplicial complexes signals can be shifted by the Dirac operator

$$\begin{bmatrix} \tilde{\mathbf{v}} \\ \tilde{\mathbf{f}} \\ \tilde{\mathbf{t}} \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{B}_1 & \mathbf{0} \\ \mathbf{B}_1^\top & \mathbf{0} & \mathbf{B}_2 \\ \mathbf{0} & \mathbf{B}_2^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{v} \\ \mathbf{f} \\ \mathbf{t} \end{bmatrix} = \begin{bmatrix} \mathbf{B}_1 \mathbf{f} \\ \mathbf{B}_1^\top \mathbf{v} + \mathbf{B}_2 \mathbf{t} \\ \mathbf{B}_2^\top \mathbf{t} \end{bmatrix}.$$

- Dirac filters

$$\mathbf{H} = \underbrace{\sum_{j=1}^J h_{j,l} \mathbf{D}_l^j}_{\mathbf{H}_l} + \underbrace{\sum_{j=1}^J h_{j,u} \mathbf{D}_u^j}_{\mathbf{H}_u} + \underbrace{h_h \tilde{\mathbf{Q}}}_{\mathbf{H}_h}$$

$\Rightarrow \tilde{\mathbf{Q}}$ approximates the orthogonal projector onto the harmonic subspace

Generalized Simplicial Neural Networks

- Considering multiple Dirac-based simplicial filters and adding a non-linearity, we obtain the generalized layer:

$$\mathbf{V}_{\ell+1} = \sigma \left(\sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_0^p \mathbf{V}_{\ell} \mathbf{W}_{\ell,2p}^{(1)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_0^p \mathbf{B}_1 \mathbf{F}_{\ell} \mathbf{W}_{\ell,2p+1}^{(1)} + \hat{\mathbf{Q}}_0 \mathbf{V}_{\ell} \mathbf{W}_{l,h} \right)$$

Generalized Simplicial Neural Networks

- Considering multiple Dirac-based simplicial filters and adding a non-linearity, we obtain the generalized layer:

$$\mathbf{V}_{\ell+1} = \sigma \left(\sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_0^p \mathbf{V}_\ell \mathbf{W}_{\ell,2p}^{(1)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_0^p \mathbf{B}_1 \mathbf{F}_\ell \mathbf{W}_{\ell,2p+1}^{(1)} + \hat{\mathbf{Q}}_0 \mathbf{V}_\ell \mathbf{W}_{l,h} \right)$$

$$\begin{aligned} \mathbf{F}_{\ell+1} = \sigma \left(\sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_{1,l}^p \mathbf{F}_\ell \mathbf{W}_{\ell,2p}^{(1)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_{1,\ell}^p \mathbf{B}_1^T \mathbf{V}_\ell \mathbf{W}_{\ell,2p+1}^{(1)} \right. \\ \left. + \sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_{1,u}^p \mathbf{F}_\ell \mathbf{W}_{\ell,2p}^{(2)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_{1,u}^p \mathbf{B}_2 \mathbf{T}_\ell \mathbf{W}_{\ell,2p+1}^{(2)} + \hat{\mathbf{Q}}_1 \mathbf{F}_\ell \mathbf{W}_{l,h} \right) \end{aligned}$$

Generalized Simplicial Neural Networks

- Considering multiple Dirac-based simplicial filters and adding a non-linearity, we obtain the generalized layer:

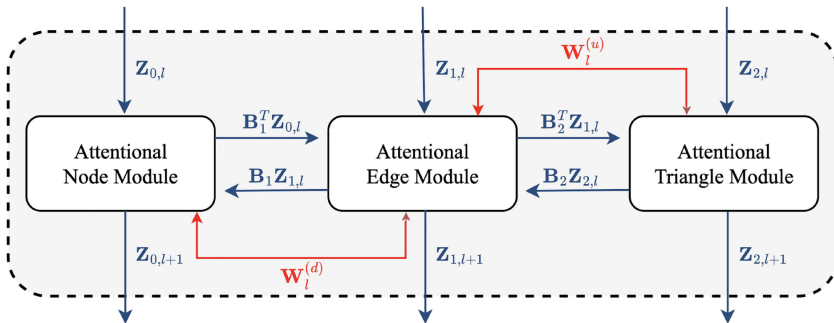
$$\mathbf{V}_{\ell+1} = \sigma \left(\sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_0^p \mathbf{V}_\ell \mathbf{W}_{\ell,2p}^{(1)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_0^p \mathbf{B}_1 \mathbf{F}_\ell \mathbf{W}_{\ell,2p+1}^{(1)} + \hat{\mathbf{Q}}_0 \mathbf{V}_\ell \mathbf{W}_{l,h} \right)$$

$$\begin{aligned} \mathbf{F}_{\ell+1} = \sigma \left(\sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_{1,l}^p \mathbf{F}_\ell \mathbf{W}_{\ell,2p}^{(1)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_{1,\ell}^p \mathbf{B}_1^T \mathbf{V}_\ell \mathbf{W}_{\ell,2p+1}^{(1)} \right. \\ \left. + \sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_{1,u}^p \mathbf{F}_\ell \mathbf{W}_{\ell,2p}^{(2)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_{1,u}^p \mathbf{B}_2 \mathbf{T}_\ell \mathbf{W}_{\ell,2p+1}^{(2)} + \hat{\mathbf{Q}}_1 \mathbf{F}_\ell \mathbf{W}_{l,h} \right) \end{aligned}$$

$$\mathbf{T}_{\ell+1} = \sigma \left(\sum_{p=1}^{\lfloor J/2 \rfloor} \mathbf{L}_2^p \mathbf{T}_\ell \mathbf{W}_{\ell,2p}^{(2)} + \sum_{p=0}^{\lceil J/2 \rceil - 1} \mathbf{L}_2^p \mathbf{B}_2^T \mathbf{F}_\ell \mathbf{W}_{\ell,2p+1}^{(2)} + \hat{\mathbf{Q}}_2 \mathbf{T}_\ell \mathbf{W}_{l,h} \right)$$

- The data at different orders is processed via a [weight sharing](#) mechanism

GSAN Layer



- The Laplacian coefficients at different orders are learnt using a similar attention mechanism as in SAN

Permutation-Equivariance and Simplicial Awareness

Permutation equivariance. GSAN is equivariant to permutations.

Simplicial awareness. GSAN is **simplicial-aware**, i.e., it respects the structure of the simplicial complex **preserving** relationships between nodes, edges, triangles, across dimensions.

TABLE II
MISSING DATA IMPUTATION TEST ACCURACY

%Miss/Order N_k	Method	0 352	1 1474	2 3285	3 5019	4 5559	5 4547
10%	SNN [29]	91 $\pm$ 0.3	91 $\pm$ 0.2	91 $\pm$ 0.2	91 $\pm$ 0.2	91 $\pm$ 0.2	90 $\pm$ 0.4
	SCNN [34]	91 $\pm$ 0.4	91 $\pm$ 0.2	91 $\pm$ 0.2	91 $\pm$ 0.2	91 $\pm$ 0.2	91 $\pm$ 0.2
	SCNN (ours)	90 $\pm$ 0.3	91 $\pm$ 0.3	91 $\pm$ 0.3	93 $\pm$ 0.2	92 $\pm$ 0.2	94 $\pm$ 0.1
	SAT [39]	18 $\pm$ 0.0	31 $\pm$ 0.0	28 $\pm$ 0.1	34 $\pm$ 0.1	53 $\pm$ 0.1	55 $\pm$ 0.1z
	GSAN	91 $\pm$ 0.4	95 $\pm$ 1.9	95 $\pm$ 1.9	97 $\pm$ 1.6	98 $\pm$ 0.9	98 $\pm$ 0.7
20%	SNN [29]	81 $\pm$ 0.6	82 $\pm$ 0.3	81 $\pm$ 0.6	82 $\pm$ 0.3	81 $\pm$ 0.6	82 $\pm$ 0.5
	SCNN [34]	81 $\pm$ 0.7	82 $\pm$ 0.3	81 $\pm$ 0.7	82 $\pm$ 0.3	81 $\pm$ 0.7	83 $\pm$ 0.3
	SCNN (ours)	81 $\pm$ 0.6	83 $\pm$ 0.7	81 $\pm$ 0.6	88 $\pm$ 0.4	86 $\pm$ 0.7	89 $\pm$ 0.6
	SAT [39]	18 $\pm$ 0.0	30 $\pm$ 0.0	29 $\pm$ 0.1	35 $\pm$ 0.1	50 $\pm$ 0.1	58 $\pm$ 0.1
	GSAN	82 $\pm$ 0.8	91 $\pm$ 2.4	82 $\pm$ 0.8	96 $\pm$ 0.4	96 $\pm$ 1.3	97 $\pm$ 0.9
30%	SNN [29]	72 $\pm$ 0.6	73 $\pm$ 0.4	81 $\pm$ 0.6	82 $\pm$ 0.3	81 $\pm$ 0.6	73 $\pm$ 0.5
	SCNN [34]	72 $\pm$ 0.5	73 $\pm$ 0.4	81 $\pm$ 0.7	82 $\pm$ 0.3	81 $\pm$ 0.7	74 $\pm$ 0.3
	SCNN (ours)	72 $\pm$ 0.6	76 $\pm$ 0.6	81 $\pm$ 0.6	82 $\pm$ 1.2	80 $\pm$ 0.7	86 $\pm$ 0.8
	SAT [39]	19 $\pm$ 0.0	33 $\pm$ 0.1	25 $\pm$ 0.1	33 $\pm$ 0.0	47 $\pm$ 0.1	53 $\pm$ 0.1
	GSAN	75 $\pm$ 2.1	89 $\pm$ 2.1	82 $\pm$ 0.8	94 $\pm$ 0.4	95 $\pm$ 0.5	96 $\pm$ 0.5
40%	SNN [29]	63 $\pm$ 0.7	64 $\pm$ 0.3	81 $\pm$ 0.6	82 $\pm$ 0.3	81 $\pm$ 0.6	65 $\pm$ 0.3
	SCNN [34]	63 $\pm$ 0.6	64 $\pm$ 0.3	81 $\pm$ 0.7	82 $\pm$ 0.3	81 $\pm$ 0.7	65 $\pm$ 0.2
	SCNN (ours)	63 $\pm$ 0.7	67 $\pm$ 1.1	81 $\pm$ 0.6	79 $\pm$ 1.0	74 $\pm$ 1.1	83 $\pm$ 0.9
	SAT [39]	20 $\pm$ 0.0	29 $\pm$ 0.0	22 $\pm$ 0.0	43 $\pm$ 0.1	51 $\pm$ 0.1	50 $\pm$ 0.1
	GSAN	67 $\pm$ 1.9	85 $\pm$ 2.8	82 $\pm$ 0.8	91 $\pm$ 0.9	93 $\pm$ 1.1	95 $\pm$ 1.6
50%	SNN [29]	54 $\pm$ 0.7	55 $\pm$ 0.5	81 $\pm$ 0.6	82 $\pm$ 0.3	81 $\pm$ 0.6	56 $\pm$ 0.3
	SCNN [34]	54 $\pm$ 0.6	55 $\pm$ 0.4	81 $\pm$ 0.7	82 $\pm$ 0.3	81 $\pm$ 0.7	56 $\pm$ 0.3
	SCNN (ours)	55 $\pm$ 0.9	60 $\pm$ 1.1	81 $\pm$ 0.6	71 $\pm$ 1.3	68 $\pm$ 1.3	79 $\pm$ 2.0
	SAT [39]	19 $\pm$ 0.0	30 $\pm$ 0.1	22 $\pm$ 0.0	32 $\pm$ 0.1	43 $\pm$ 0.0	48 $\pm$ 0.1
	GSAN	61 $\pm$ 1.9	79 $\pm$ 4.3	82 $\pm$ 0.8	88 $\pm$ 1.5	92 $\pm$ 0.7	94 $\pm$ 1.1

Self-supervision with topologies

Self-supervised Contrastive Learning

- ▶ So far, we impose the **topological** structure **within** architectures
 - ⇒ Training remains **supervised** and **generalist**

Self-supervised Contrastive Learning

- ▶ So far, we impose the **topological** structure **within** architectures
 - ⇒ Training remains **supervised** and **generalist**
- ▶ Here, we want to impose the topological-structure in the entire machine learning pipeline
 - ⇒ **data** generation, **architecture**, **loss function**
 - ⇒ Via a self-supervised contrastive learning perspective

Self-supervised Contrastive Learning

- ▶ So far, we impose the **topological** structure **within** architectures
 - ⇒ Training remains **supervised** and **generalist**
- ▶ Here, we want to impose the topological-structure in the entire machine learning pipeline
 - ⇒ **data** generation, **architecture**, **loss function**
 - ⇒ Via a self-supervised contrastive learning perspective

Key Idea: Train a model to **learn** representations by **pulling** similar examples together and **pushing** different ones apart.

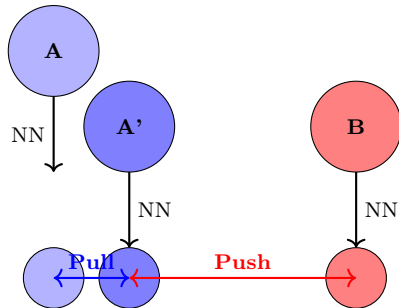
Self-supervised Contrastive Learning

- ▶ So far, we impose the **topological** structure **within** architectures
 - ⇒ Training remains **supervised** and **generalist**
- ▶ Here, we want to impose the topological-structure in the entire machine learning pipeline
 - ⇒ **data** generation, **architecture**, **loss function**
 - ⇒ Via a self-supervised contrastive learning perspective

Key Idea: Train a model to **learn** representations by **pulling** similar examples together and **pushing** different ones apart.

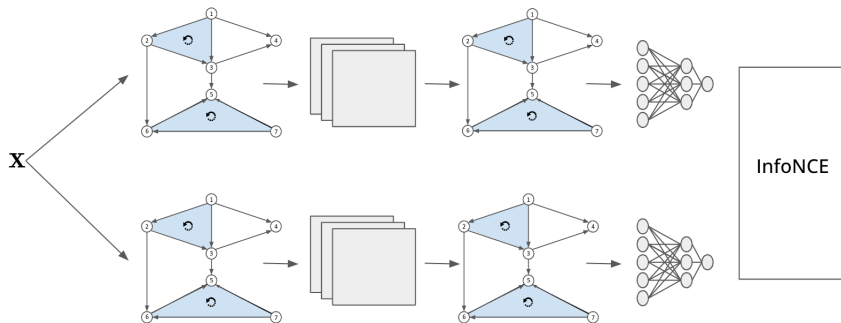
Steps:

- ▶ Given **input A**, generate two augmented versions **input A'** and **input B**
- ▶ Encode these inputs into respective deep representations.
- ▶ Contrastive loss:
 - ▶ **Similar (positive) pairs** → pulled closer.
 - ▶ **Different (negative) pairs** → pushed apart.



Simplicial Contrastive Learning (SCL)

- Create **positive** and **negative** simplicial signal augmentations
- A **Simplicial Neural Network** encodes the data and **generates embeddings**
- Embeddings of **positives** are **mapped close** to each other and **negatives** further **apart**



Mollers, Immer, Fortuin, Isufi, Hodge-aware contrastive learning, ICASSP, 2024

The InfoNCE

Similarity: anchor embedding and a positive sample

$$\mathcal{L}_{\text{infoNCE}} = - \sum_{(\mathbf{f}'_i, \mathbf{f}'_j) \in \mathbb{P}} \log \left(\frac{e^{\text{sim}(\mathbf{z}'_i, \mathbf{z}'_j) / \tau}}{\sum_{m=1}^M e^{\text{sim}(\mathbf{z}'_i, \mathbf{z}_m) / \tau}} \right)$$

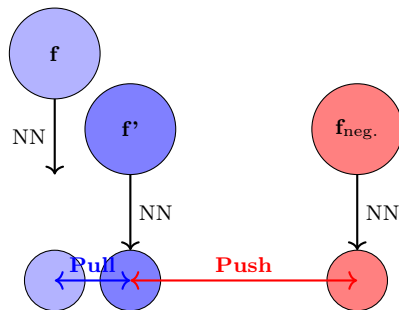
Sum over all positive pairs

Similarity: anchor embedding and M negatives

Topology-aware Organized Embedding

- Create topology-aware **positive** and **negative** signal augmentations
- Compute a **weighted** embedding cosine distance as a similarity measure

$$\mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m) = \gamma_H \text{CD}(\tilde{\mathbf{f}}_{H,i}, \tilde{\mathbf{f}}_{H,m}) + \gamma_G \text{CD}(\tilde{\mathbf{f}}_{G,i}, \tilde{\mathbf{f}}_{G,m}) \\ + \gamma_C \text{CD}(\tilde{\mathbf{f}}_{C,i}, \tilde{\mathbf{f}}_{C,m})$$



Topology-Aware Organized Embedding

- ▶ Create topology-aware **positive** and **negative** signal augmentations
- ▶ Compute a **weighted** embedding cosine distance as a dissimilarity measure **1-cosine similarity**

$$\mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m) = \gamma_H \text{CD}(\tilde{\mathbf{f}}_{H,i}, \tilde{\mathbf{f}}_{H,m}) + \gamma_G \text{CD}(\tilde{\mathbf{f}}_{G,i}, \tilde{\mathbf{f}}_{G,m}) \\ + \gamma_C \text{CD}(\tilde{\mathbf{f}}_{C,i}, \tilde{\mathbf{f}}_{C,m})$$

- ▶ **Normalize** weight of each negative example

$$w(\mathbf{f}_i, \mathbf{f}_m) = \frac{\mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m)}{\sum_{m=1}^M \mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m)}$$

Topology-Aware Organized Embedding

- ▶ Create topology-aware **positive** and **negative** signal augmentations
- ▶ Compute a **weighted** embedding cosine distance as a dissimilarity measure 1-cosine similarity

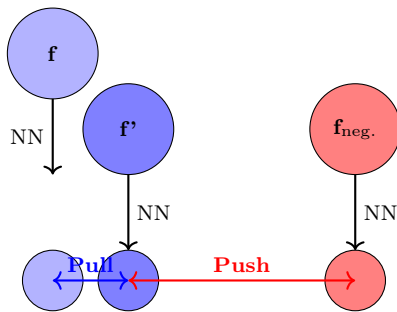
$$\mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m) = \gamma_H \text{CD}(\tilde{\mathbf{f}}_{H,i}, \tilde{\mathbf{f}}_{H,m}) + \gamma_G \text{CD}(\tilde{\mathbf{f}}_{G,i}, \tilde{\mathbf{f}}_{G,m}) \\ + \gamma_C \text{CD}(\tilde{\mathbf{f}}_{C,i}, \tilde{\mathbf{f}}_{C,m})$$

- ▶ **Normalize** weight of each negative example

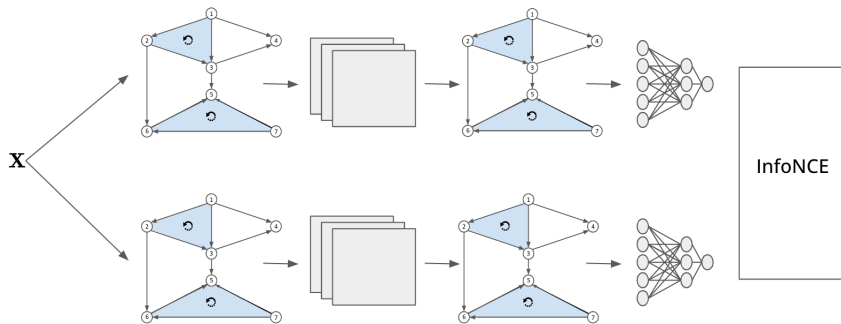
$$w(\mathbf{f}_i, \mathbf{f}_m) = \frac{\mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m)}{\sum_{m=1}^M \mathcal{S}(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_m)}$$

- ▶ Obtain a spectrally weighted loss that **pushes** **dissimilar examples** further **apart**

$$\mathcal{L}_{\text{weighted}} = - \sum_{(\mathbf{f}'_i, \mathbf{f}'_j) \in \mathbb{P}} \log \left(\frac{e^{\text{sim}(\mathbf{z}'_i, \mathbf{z}'_j)/\tau}}{\sum_{m=1}^M w(\mathbf{f}_i, \mathbf{f}_m) e^{\text{sim}(\mathbf{z}'_i, \mathbf{z}_m)/\tau}} \right)$$



SCL Pipeline



- ▶ The topological information in SCL is considered in
 - ⇒ **the architecture**: convolutional, attention ...
 - ⇒ **the loss function**: organizing embedding based on signal spectral properties
- ▶ It remains open to include the **topology** in the signal **augmentation**

Topology-Aware Signal Augmentation

- ▶ Dropout is the standard augmentation method for CL (e.g. dropping edges)
- ▶ Uniformly dropping edges destroys spectral information
- ▶ Optimize the dropout probabilities based on the signal properties
- ▶ We preserve a particular (prior)

Topology-Aware Signal Augmentation

$$\mathbf{f}' := \mathbf{f} \circ \mathbf{e}; \quad \mathbf{e}_i \sim \text{Ber}(\mathbf{p}_i)$$

- ▶ Dropout is the standard augmentation method for CL (e.g. dropping edges)
- ▶ Uniformly dropping edges destroys spectral information
- ▶ Optimize the dropout probabilities based on the signal properties
- ▶ We preserve a particular (prior)

Topology-Aware Signal Augmentation

$$\mathbf{f}' := \mathbf{f} \circ \mathbf{e}; \quad \mathbf{e}_i \sim \text{Ber}(\mathbf{p}_i)$$



$$\mathcal{L}_G(\mathbf{p}) = \mathbb{E}[\|\tilde{\mathbf{f}}_G - \tilde{\mathbf{f}}'_G\|_2^2], \mathcal{L}_C(\mathbf{p}) = \mathbb{E}[\|\tilde{\mathbf{f}}_C - \tilde{\mathbf{f}}'_C\|_2^2],$$

$$\mathcal{L}_H(\mathbf{p}) = \mathbb{E}[\|\tilde{\mathbf{f}}_H - \tilde{\mathbf{f}}'_H\|_2^2]$$

- ▶ Dropout is the standard augmentation method for CL (e.g. dropping edges)
- ▶ Uniformly dropping edges destroys spectral information
- ▶ Optimize the dropout probabilities based on the signal properties
- ▶ We preserve a particular (prior)

Topology-Aware Signal Augmentation

$$\mathbf{f}' := \mathbf{f} \circ \mathbf{e}; \quad \mathbf{e}_i \sim \text{Ber}(\mathbf{p}_i)$$



$$\mathcal{L}_G(\mathbf{p}) = \mathbb{E}[\|\tilde{\mathbf{f}}_G - \tilde{\mathbf{f}}'_G\|_2^2], \mathcal{L}_C(\mathbf{p}) = \mathbb{E}[\|\tilde{\mathbf{f}}_C - \tilde{\mathbf{f}}'_C\|_2^2],$$

$$\mathcal{L}_H(\mathbf{p}) = \mathbb{E}[\|\tilde{\mathbf{f}}_H - \tilde{\mathbf{f}}'_H\|_2^2]$$



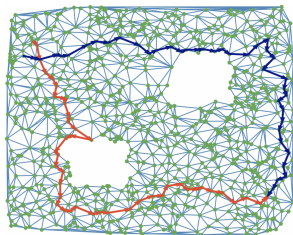
$$\min_{\mathbf{p}} \quad -\mathcal{L}_G(\mathbf{p}) + \mathcal{L}_C(\mathbf{p}) + \mathcal{L}_H(\mathbf{p})$$

$$\text{subject to} \quad \mathbf{p} \in \mathcal{G}_{\mathbf{p}} := \left\{ \mathbf{p} \mid \mathbf{p} \in [0, 1]^{N_1}, \|\mathbf{p}\|_1 \leq \epsilon_{\mathbf{p}} \right\}$$

- Dropout is the standard augmentation method for CL (e.g. dropping edges)
- Uniformly dropping edges destroys spectral information
- Optimize the dropout probabilities based on the signal properties
- We preserve a particular (prior)

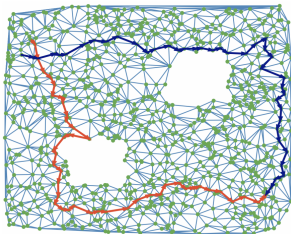
Numerical Results

- ▶ Setup: **trajectory classification tasks**
 - ⇒ Differentiate between bottom-left and top-right trajectories
 - ⇒ Distinguish clockwise from anti-clockwise moving drifters around Madagascar
- ▶ **Harmonic** embeddings capture important information
 - ⇒ Wholes in the underlying SC
 - ⇒ Trained a linear SVM on the generated representations



Numerical Results

- ▶ Setup: trajectory classification tasks
 - ⇒ Differentiate between bottom-left and top-right trajectories
 - ⇒ Distinguish clockwise from anti-clockwise moving drifters around Madagascar
- ▶ Harmonic embeddings capture important information
 - ⇒ Wholes in the underlying SC
 - ⇒ Trained a linear SVM on the generated representations



Model	Trajectory Task	Ocean Drifters
SSCL _{Spec}	97.9 $\pm$ 0.3	90.3 $\pm$ 1.4
SCNN (supervised)	95.2 $\pm$ 0.5	78.5 $\pm$ 1.1
SSCL	96.8 $\pm$ 0.4	89.1 $\pm$ 1.0
SCL _{Spec}	98.2 $\pm$ 0.4	83.1 $\pm$ 1.1
SCL	96.1 $\pm$ 0.6	81.6 $\pm$ 1.6
SCL _{low}	91.0 $\pm$ 0.2	77.1 $\pm$ 1.2

- ▶ Relevant topological information is powerful via a self-supervised approach
- ▶ Spectral augmentations improve over uniform augmentations

Learning Activity: What Does Topological Deep Learning Actually Learn?

Topological Deep Learning constructs models that operate on multi-order signals (nodes, edges, triangles) while embedding topological structure directly into learning. This affects not only the architecture, but also the representation space.

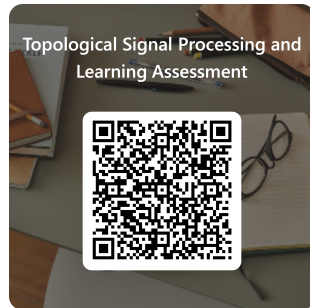
- ▶ **Q6.2:** In topological deep learning, what is the primary role of the topology (e.g., simplicial complex structure) during learning?

- ▶ **Instructions**

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

- ▶ **Logistics**

- ⇒ **Time:** 8 min
- ⇒ **Goal:** Understand how topology acts as an inductive bias across architectures and representations
- ⇒ **Product:** One submitted response per student + justification to peers



<https://forms.office.com/>

Further Reading

- ▶ Elvin Isufi, Geert Leus, Baltasar Beferull-Lozano, Sergio Barbarossa, Paolo Di Lorenzo, *Topological signal processing and learning: Recent advances and future challenges*, Signal Processing, Volume 233, 2025, 109930, ISSN 0165-1684, <https://doi.org/10.1016/j.sigpro.2025.109930>
- ▶ Michael T. Schaub, Yu Zhu, Jean-Baptiste Seby, T. Mitchell Roddenberry, Santiago Segarra, *Signal processing on higher-order networks: Livin' on the edge... and beyond*, Signal Processing, Volume 187, 2021, 108149, ISSN 0165-1684, <https://doi.org/10.1016/j.sigpro.2021.108149>
- ▶ Mathilde Papillon, Sophia Sanborn, Mustafa Hajij, Nina Miolane, *Architectures of Topological Deep Learning: A Survey of Message-Passing Topological Neural Networks*, 2024, ArXiv, <https://arxiv.org/abs/2304.10031>
- ▶ Maosheng Yang, Elvin Isufi, and Geert Leus. *Simplicial convolutional neural networks*. ICASSP 2022, <https://doi.org/10.1109/ICASSP43922.2022.9746017>
- ▶ Maosheng Yang, Geert Leus, and Elvin Isufi. *Hodge-aware convolutional learning on simplicial complexes*. Transactions on Machine Learning Research (2025), <https://openreview.net/forum?id=Nm5sp09Q25>
- ▶ Lorenzo Giusti, Claudio Battiloro, Paolo Di Lorenzo, Stefania Sardellitti, Sergio Barbarossa, *Simplicial Attention Neural Networks*, arXiv:2203.07485, 2022, <https://arxiv.org/abs/2203.07485>
- ▶ Alexander Möllers, Alexander Immer, Vincent Fortuin, and Elvin Isufi, *Hodge-Aware Contrastive Learning*, ICASSP 2024, <https://ieeexplore.ieee.org/abstract/document/10447015>

Video Material

- ▶ Sundeep Prabhakar Chepuri, *Simple Yet Powerful Graph-aware and Simplicial-aware Neural Models*, <https://www.youtube.com/watch?v=CNishpvy11A>
- ▶ Paolo Di Lorenzo, *Task-driven Topology Inference for Signal processing and Learning over Topological Domains*, <https://www.youtube.com/watch?v=PPy8SJpmkCU>
- ▶ Mathilde Papillon, *Introduction to Topological Deep Learning*, <https://www.youtube.com/watch?v=N67GABKImZA>
- ▶ Santiago Segarra *Principled Simplicial Neural Networks for Trajectory Prediction*, <https://www.youtube.com/watch?v=bjgfi0Y5xFM>